KU LEUVEN

FACULTEIT INDUSTRIELE INGENIEURSWETENSCHAPPEN

TECHNOLOGIECAMPUS GENT

Geavanceerde Computerarchitectuur

Report 5
MEI-ecfs

Charlotte BERT
Sander HAUSTRAETE

Academiejaar 2025 - 2026

1 Context

In dit labo wordt onderzocht hoe synchrone versus asynchrone uitvoering van een GPU kernel de uitvoeringstijd en parallelisme beïnvloedt. Er wordt nagegaan hoe het overlappen van CPU- en GPU-werk via asynchrone kernel launches en geheugentransfers de totale uitvoeringstijd kan verkleinen. De verschillen in structuur tussen een synchrone en asynchrone uitvoer worden bekeken via NVIDIA Nsight Systems.

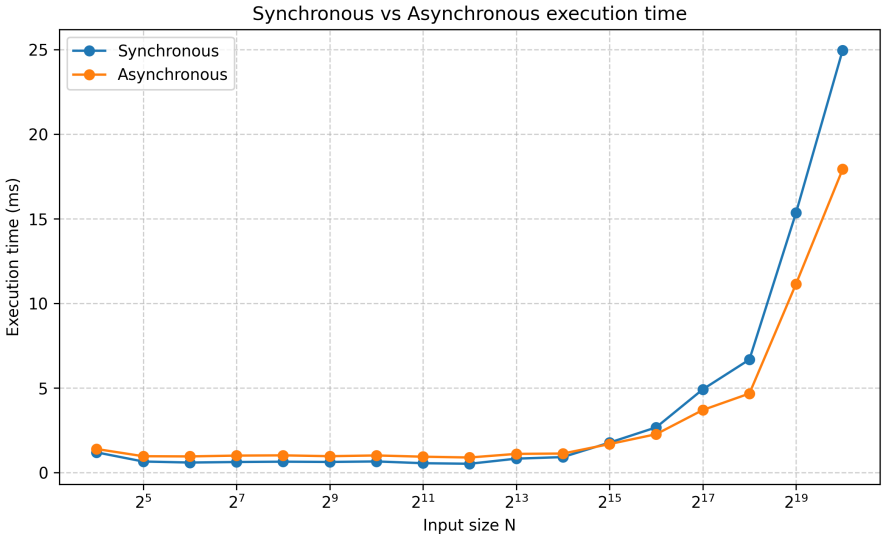
2 Methodologie

Eerst werd de max reduction kernel van labo 2 aangepast zodanig dat deze een extra argument kon binnennemen om het type operatie te specifiëren. Op basis van dit argument kan dezelfde kernel gebruikt worden voor het berekenen van de som, het product, het minimum of het maximum van een array. Vervolgens werd een CPU-functie geschreven om arrays te genereren, waarbij de gegenereerde waarden afhankelijk zijn van de uit te voeren operatie (zo zijn de waarden binnen $[0, 1]$ bij het product om een overflow te vermijden). Daarna werden er twee functies opgesteld, die elk overeenkomt met een andere uitvoeringsmethode. In de synchrone versie worden de arrays sequentieel gegenereerd en verwerkt, waarbij na elke kerneluitvoering expliciet gesynchroniseerd wordt. In de asynchrone versie wordt er gebruik gemaakt van CUDA streams en asynchrone geheugentransfers, zodat de CPU steeds de volgende invoer kan genereren terwijl de GPU de kernel uitvoert. Tot slot werden beide methoden uitgevoerd en geëvalueerd voor verschillende invoergroottes. Voor elke invoergrootte werd de uitvoering 200 keer herhaald en werd hier de gemiddelde uitvoeringstijd van bepaald.

3 Observatie/ resultaat

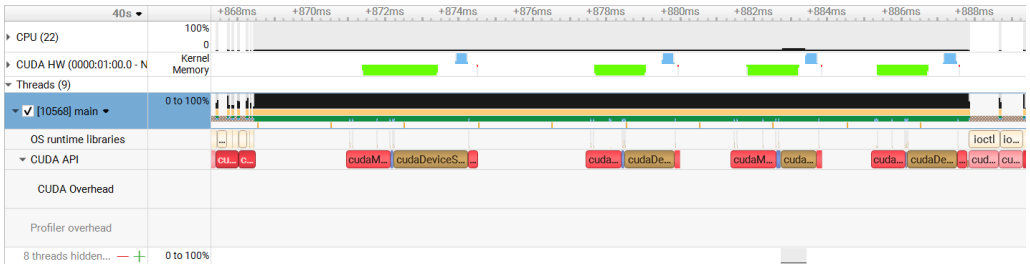
```
 Sync  n:          16 time:     1.18578 ms
Async  n:          16 time:     1.38437 ms
-----------------
 Sync  n:          32 time:    0.644143 ms
Async  n:          32 time:    0.957162 ms
-----------------
 Sync  n:          64 time:    0.583923 ms
Async  n:          64 time:    0.945998 ms
-----------------
 Sync  n:         128 time:    0.615188 ms
Async  n:         128 time:    0.994435 ms
-----------------
 Sync  n:         256 time:    0.634735 ms
Async  n:         256 time:     1.00893 ms
-----------------
 Sync  n:         512 time:    0.618229 ms
Async  n:         512 time:    0.959118 ms
-----------------
 Sync  n:        1024 time:    0.648753 ms
Async  n:        1024 time:     1.00297 ms
-----------------
 Sync  n:        2048 time:    0.544927 ms
Async  n:        2048 time:    0.929392 ms
-----------------
 Sync  n:        4096 time:    0.511309 ms
Async  n:        4096 time:    0.881619 ms
-----------------
 Sync  n:        8192 time:    0.813948 ms
Async  n:        8192 time:     1.09597 ms
-----------------
 Sync  n:       16384 time:    0.906863 ms
Async  n:       16384 time:     1.11592 ms
-----------------
 Sync  n:       32768 time:     1.76283 ms
Async  n:       32768 time:     1.67611 ms
-----------------
 Sync  n:       65536 time:     2.66463 ms
Async  n:       65536 time:     2.26022 ms
-----------------
 Sync  n:      131072 time:     4.91195 ms
Async  n:      131072 time:     3.69056 ms
-----------------
 Sync  n:      262144 time:      6.6746 ms
Async  n:      262144 time:     4.66516 ms
-----------------
 Sync  n:      524288 time:     15.3581 ms
Async  n:      524288 time:       11.15 ms
-----------------
 Sync  n:     1048576 time:     24.9721 ms
Async  n:     1048576 time:     17.9311 ms
-----------------
```

Codefragment 1: Terminal output



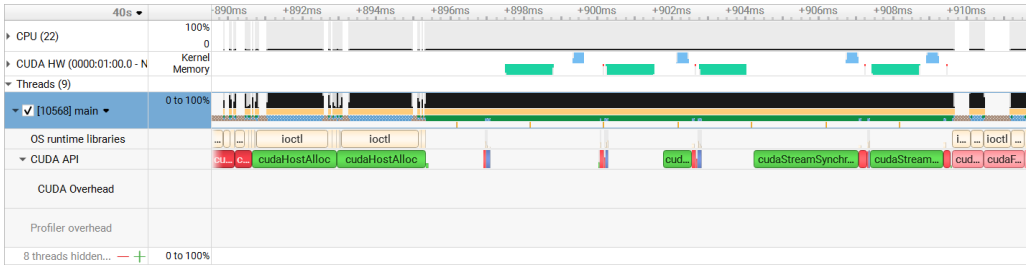
Figuur 1: Synchrone vs Asynchrone methode uitvoeringstijden

Zoals te zien op codefragment 1 en figuur 1 is de asynchrone methode bij kleinere invoergroottes lichtjes trager dan de synchrone methode. Dit is te wijten aan de extra overhead bij asynchrone uitvoering, zoals het gebruik van CUDA streams en asynchrone geheugentransfers. Vanaf een invoergrootte van 32768 (2^{15}) elementen wordt de asynchrone methode echter sneller dan de synchrone methode. Bij grotere invoergroottes wordt de rekentijd op de GPU dominant, waardoor de CPU- en GPU-activiteiten beter overlappen. Hierdoor kan de CPU nieuwe data voorbereiden terwijl de GPU bezig is met rekenen, wat resulteert in een lagere totale uitvoeringstijd. De relatieve impact van de overhead neemt dus af naarmate de invoergrootte toeneemt, terwijl de voordelen van overlappende uitvoering toenemen.



Figuur 2: Synchrone uitvoering bij invoergroote 2^{20} via Nsight

Deze structuur is duidelijk te zien in de NVIDIA Nsight Systems profiler (figuren 2 en 3). Bij de synchrone uitvoering (figuur 2) is te zien dat GPU-activiteiten (kerneluitvoering en geheugentransfers) gescheiden worden door periodes waarin de GPU inactief is. Tijdens deze periodes wordt de volledige array op de CPU gegenereerd, waardoor de GPU moet wachten voordat de volgende kernel kan starten.



Figuur 3: Asynchrone uitvoering bij invoergroote 2^{20} via Nsight

Bij de asynchrone uitvoering (figuur 3) zijn deze inactieve periodes aanzienlijk kleiner. Hier overlappen de GPU- en CPU-activiteiten: terwijl de GPU de huidige kernel uitvoert, begint de CPU met het genereren van de invoer voor de volgende operatie. Hierdoor kan de GPU sneller opeenvolgende kernels uitvoeren.

4 Conclusie

In dit labo werd aangetoond dat asynchrone uitvoering van GPU-kernels een grote tijdswinst kan opleveren voor grotere invoergroottes. Door gebruik te maken van CUDA streams en asynchrone geheugentransfers kunnen de CPU- en GPU-activiteiten overlappen, waardoor de GPU minder tijd in een inactieve toestand doorbrengt.

Echter, bij kleinere invoergroottes weegt de extra overhead van asynchrone uitvoering zwaarder door dan de mogelijke winst door overlap, waardoor hier de synchrone implementatie lichtjes sneller blijft. Dit toont aan dat asynchrone uitvoering pas voordelig is wanneer de rekentijd voldoende groot is om de bijkomende overhead te compenseren.

```
#include <iostream>
#include <cuda_runtime.h>
#include <vector>
#include <chrono>
#include <cfloat>
#include <iomanip>

#define OP_SUM 0
#define OP_PROD 1
#define OP_MIN 2
#define OP_MAX 3

// kernels

__global__ void reduction_kernel(float *input, float *block_out, int n, int op)
     {
    __shared__ float sdata[1024];

    int tid = threadIdx.x;
    int idx = threadIdx.x + blockIdx.x * blockDim.x;

    //load global memory in shared memory
    if (idx < n) {
        sdata[tid] = input[idx];
    } else {
        if (op == OP_SUM) sdata[tid] = 0.0f;
        if (op == OP_PROD) sdata[tid] = 1.0f;
        if (op == OP_MIN) sdata[tid] = FLT_MAX;
        if (op == OP_MAX) sdata[tid] = -FLT_MAX;
    }

    __syncthreads();

    for (int step = 1; step < blockDim.x; step *= 2) {
        int i = 2 * step * tid;
        int other_i = i + step;
        if (i + step < blockDim.x) {
            if (op == OP_SUM) sdata[i] += sdata[other_i];
            else if (op == OP_PROD) sdata[i] *= sdata[other_i];
            else if (op == OP_MIN) sdata[i] = (sdata[i] > sdata[other_i]) ?
    sdata[other_i]: sdata[i];
            else if (op == OP_MAX) sdata[i] = (sdata[i] < sdata[other_i]) ?
    sdata[other_i]: sdata[i];
        }

        __syncthreads();
    }

    if (tid == 0) {
        block_out[blockIdx.x] = sdata[0];
```

```
    }

}


// CPU Functions
void generate_array(float* arr, int n, int op) {
    for (int i = 0; i < n; i++) {
        switch(op) {
            case OP_SUM:
            case OP_MIN:
            case OP_MAX:
                arr[i] = i;
                break;
            case OP_PROD:
                //arr[i] = (i+1) * 1.0f/n; // resulteert in prod = 0
                arr[i] = 0.99f + 0.01f * ((i % 10) / 10.0f);
                break;
            default:
                break;
        }
    }
}

std::string opToString(int op) {
    std::string ret;
    switch(op) {
        case OP_SUM:
            ret = "SUM";
            break;

        case OP_PROD:
            ret = "PROD";
            break;

        case OP_MIN:
            ret = "MIN";
            break;

        case OP_MAX:
            ret = "MAX";
            break;
    }
    return ret;
}

float run_sync(int n) {
    const int ops = 4;
    const int threadsPerBlock = 256;
    int blocks = (n + threadsPerBlock - 1) / threadsPerBlock;

```

```
    float *d_input, *d_partial;
    cudaMalloc(&d_input, n*sizeof(float));
    cudaMalloc(&d_partial, blocks*sizeof(float));

    std::vector<float> h_input(n);

    const auto t0 = std::chrono::steady_clock::now();

    for (int op = 0; op < ops; op++) {
        // generate arrays
        generate_array(h_input.data(), n, op);

        // reductions
        cudaMemcpy(d_input, h_input.data(), n*sizeof(float), cudaMemcpyHostToDevice);

        reduction_kernel<<<blocks, threadsPerBlock>>>(d_input, d_partial, n, op);
        cudaDeviceSynchronize();

        std::vector<float> partial(blocks);
        cudaMemcpy(partial.data(), d_partial, blocks * sizeof(float), cudaMemcpyDeviceToHost);

        float result = partial[0];
        for (int i = 1; i < blocks; i++) {
            switch (op) {
                case OP_SUM:
                    result += partial[i];
                    break;

                case OP_PROD:
                    result *= partial[i];
                    break;

                case OP_MIN:
                    result = std::min(result, partial[i]);
                    break;

                case OP_MAX:
                    result = std::max(result, partial[i]);
                    break;
            }
        }

        //std::cout << "sync op " << opToString(op) << " result = " << result << std::endl;
    }

    cudaDeviceSynchronize();
    const auto t1 = std::chrono::steady_clock::now();
```

```
146
147     double ms = std::chrono::duration<double, std::milli>(t1 - t0).count();
148     //std::cout << "Total synchronous time: " << ms << "ms" << std::endl;
149
150     cudaFree(d_input);
151     cudaFree(d_partial);
152
153     return ms;
154 }
155
156 float run_async(int n) {
157     const int ops = 4;
158     const int threadsPerBlock = 256;
159     int blocks = (n + threadsPerBlock - 1) / threadsPerBlock;
160
161     float *d_input, *d_partial;
162     cudaMalloc(&d_input, n*sizeof(float));
163     cudaMalloc(&d_partial, blocks*sizeof(float));
164
165     float *h_buff[2]; // buffer arrays to enable better parallelism
166     cudaMallocHost(&h_buff[0], n*sizeof(float));
167     cudaMallocHost(&h_buff[1], n*sizeof(float));
168
169     cudaStream_t stream;
170     cudaStreamCreate(&stream);
171
172     const auto t0 = std::chrono::steady_clock::now();
173
174     //generate first array
175     generate_array(h_buff[0], n, OP_SUM);
176
177     for (int op = 0; op < ops; op++) {
178         int cur_buf = op % 2;
179         int next_buf = (op + 1) % 2;
180
181         cudaMemcpyAsync(d_input, h_buff[cur_buf], n*sizeof(float),
    cudaMemcpyHostToDevice, stream);
182
183         reduction_kernel<<<blocks, threadsPerBlock, 0, stream>>>(d_input,
    d_partial, n, op);
184
185         //CPU prepares next input while GPU working
186         if (op + 1 < ops) {
187             generate_array(h_buff[next_buf], n, op + 1);
188         }
189
190         cudaStreamSynchronize(stream);
191
192         std::vector<float> partial(blocks);
193         cudaMemcpy(partial.data(), d_partial, blocks*sizeof(float),
    cudaMemcpyDeviceToHost);
```

```
194
195         float result = partial[0];
196         for (int i = 1; i < blocks; i++) {
197             switch (op) {
198                 case OP_SUM:
199                     result += partial[i];
200                     break;
201
202                 case OP_PROD:
203                     result *= partial[i];
204                     break;
205
206                 case OP_MIN:
207                     result = std::min(result, partial[i]);
208                     break;
209
210                 case OP_MAX:
211                     result = std::max(result, partial[i]);
212                     break;
213             }
214         }
215
216         //std::cout << "async op " << opToString(op) << " result = " << result
    << std::endl;
217     }
218
219     cudaStreamSynchronize(stream);
220
221     const auto t1 = std::chrono::steady_clock::now();
222
223     double ms = std::chrono::duration<double, std::milli>(t1 - t0).count();
224     //std::cout << "Total asynchronous time: " << ms << " ms" << std::endl;
225
226     cudaStreamDestroy(stream);
227     cudaFreeHost(h_buff[0]);
228     cudaFreeHost(h_buff[1]);
229     cudaFree(d_input);
230     cudaFree(d_partial);
231
232     return ms;
233 }
234
235
236 int main(void) {
237     int n_sizes[] = {16, 32, 64, 128, 256, 512, 1024, 2048, 4096, 1 << 13, 1 <<
     14, 1 << 15, 1 << 16, 1 << 17, 1 << 18, 1 << 19, 1 << 20};
238     int iterations = 200;
239
240     for (int n: n_sizes) {
241         float ms_sync_total = 0.0f;
242         float ms_async_total = 0.0f;
```

```
243
244         for (int i = 0; i < iterations; i++){
245             float ms_sync = 0.0f;
246             float ms_async = 0.0f;
247
248             ms_sync = run_sync(n);
249
250             ms_async = run_async(n);
251
252             ms_sync_total += ms_sync;
253             ms_async_total += ms_async;
254         }
255
256         float ms_sync_avg = ms_sync_total / iterations;
257         float ms_async_avg = ms_async_total / iterations;
258
259         std::cout
260             << std::setw(6) << "Sync "
261             << std::setw(3) << " n: "
262             << std::setw(10) << n
263             << std::setw(7) << " time: "
264             << std::setw(10) << ms_sync_avg
265             << std::setw(3) << " ms" << std::endl;
266
267
268         std::cout
269             << std::setw(6) << "Async "
270             << std::setw(3) << " n: "
271             << std::setw(10) << n
272             << std::setw(7) << " time: "
273             << std::setw(10) << ms_async_avg
274             << std::setw(3) << " ms" << std::endl;
275
276         std::cout << "-----------------" << std::endl;
277     }
278 }
```

Codefragment 2: Code labo 5

FACULTEIT INDUSTRIELE INGENIEURSWETENSCHAPPEN
TECHNOLOGIECAMPUS GENT
Gebroeders De Smetstraat 1
9000 GENT, België
tel. + 32 92 65 86 10
fax + 32 92 25 62 69
iiw.gent@kuleuven.be
www.iiw.kuleuven.be

LID VAN ASSOCIATIE KU LEUVEN